Projet_apprentissage

May 26, 2025

1 Reinforcement Learning Project : N7Craft

2 Training a Minecraft Jump Agent with Mineflayer & RL

A Gym-style Env, PPO Training Loop, and Evaluation on Custom Jump Maps
Briefly outline the goal of this notebook and the high-level flow:
- Connect Mineflayer bot to a custom jump map server
- Wrap it in a Gym environment
- Train an RL agent (e.g. PPO)
- Evaluate performance

2.1 Environment Setup & Imports

Launch the Minecraft Server (version 1.19.4 if you want to connect to it as a player)

[1]:
```
import subprocess

# Start in background, no console output
subprocess.Popen(
    ['java', '-Xms1G', '-Xmx2G', '-jar', 'paper-1.19.4-550.jar'],
    cwd='minecraft-server',
    stdout=subprocess.DEVNULL,
    stderr=subprocess.STDOUT
)
```

[1]:
```
<Popen: returncode: None args: ['java', '-Xms1G', '-Xmx2G', '-jar', 'paper-1…>
```

Install and import all required libraries: 1. `pip install gym mineflayer vec3 stable-baselines3 torch matplotlib`
2. Import Python modules for Gym, Mineflayer, NumPy, etc.

[2]:
```
# Use `n` to install nodejs 18, if it's not already installed:
#!curl -fsSL https://raw.githubusercontent.com/tj/n/master/bin/n | bash -s lts␣
 ↪> /dev/null
# Now write the Node.js and Python version to the console
!node --version
!python --version
```

```
v22.13.1
Python 3.13.1
```

[3]:
```
!pip install javascript
```

```
Requirement already satisfied: javascript in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(1!1.2.2)
```

[4]:
```
!pip install gym vec3 javascript stable-baselines3 torch matplotlib
```

```
Requirement already satisfied: gym in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(0.26.2)
Requirement already satisfied: vec3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (0.1.0)
Requirement already satisfied: javascript in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(1!1.2.2)
Requirement already satisfied: stable-baselines3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.6.0)
Requirement already satisfied: torch in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.7.0)
Requirement already satisfied: matplotlib in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages
(3.10.0)
Requirement already satisfied: numpy>=1.18.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gym) (2.2.2)
Requirement already satisfied: cloudpickle>=1.2.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gym) (3.1.1)
Requirement already satisfied: gym_notices>=0.0.4 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gym) (0.0.8)
Requirement already satisfied: gymnasium<1.2.0,>=0.29.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
stable-baselines3) (1.1.1)
Requirement already satisfied: pandas in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
stable-baselines3) (2.2.3)
Requirement already satisfied: filelock in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (4.12.2)
Requirement already satisfied: sympy>=1.13.3 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
```

```
torch) (1.13.3)
Requirement already satisfied: networkx in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.4.2)
Requirement already satisfied: jinja2 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (3.1.6)
Requirement already satisfied: fsspec in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (2025.2.0)
Requirement already satisfied: setuptools in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
torch) (75.8.0)
Requirement already satisfied: farama-notifications>=0.0.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
gymnasium<1.2.0,>=0.29.1->stable-baselines3) (0.0.4)
Requirement already satisfied: contourpy>=1.0.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
```

```
jinja2->torch) (3.0.2)
Requirement already satisfied: pytz>=2020.1 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
pandas->stable-baselines3) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from
pandas->stable-baselines3) (2025.1)
```

2.1.1 Test imports

[5]:
```
from javascript import require, On, Once, AsyncTask, once, off
import time, math, os
import numpy as np
import gymnasium as gym
from gymnasium import spaces
```

```
[JSE] (node:24408) [DEP0040] DeprecationWarning: The `punycode` module is
deprecated. Please use a userland alternative instead.

[JSE] (Use `node --trace-deprecation …` to show where the warning was created)

AggregateError [ECONNREFUSED]:

    at internalConnectMultiple (node:net:1139:18)

    at afterConnectMultiple (node:net:1712:7) {

  code: 'ECONNREFUSED',

  [errors]: [

    Error: connect ECONNREFUSED ::1:25565

        at createConnectionError (node:net:1675:14)

        at afterConnectMultiple (node:net:1705:16) {

      errno: -4078,

      code: 'ECONNREFUSED',

      syscall: 'connect',

      address: '::1',

      port: 25565

    },
```

```
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
```

```
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
```

```
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
```

```
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
```

```
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
```

```
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
```

```
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
```

```
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
```

```
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
```

```
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
```

```
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
```

```
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
AggregateError [ECONNREFUSED]:
    at internalConnectMultiple (node:net:1139:18)
```

```
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}

AggregateError [ECONNREFUSED]:
```

```
    at internalConnectMultiple (node:net:1139:18)
    at afterConnectMultiple (node:net:1712:7) {
  code: 'ECONNREFUSED',
  [errors]: [
    Error: connect ECONNREFUSED ::1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '::1',
      port: 25565
    },
    Error: connect ECONNREFUSED 127.0.0.1:25565
        at createConnectionError (node:net:1675:14)
        at afterConnectMultiple (node:net:1705:16) {
      errno: -4078,
      code: 'ECONNREFUSED',
      syscall: 'connect',
      address: '127.0.0.1',
      port: 25565
    }
  ]
}
```

2.2 Bot Connection Helper

Define a utility function to spawn and login the Mineflayer bot:
- Explain that this handles network connection and event waiting

We import Mineflayer & Vec3 (Node.js libs exposed via `javascript` package)

[6]:
```
mineflayer = require('mineflayer')
Vec3 = require('vec3').Vec3
```

We define a function to spawn and login the Agent to the server in localhost.

[7]:
```
def create_bot(host='localhost', port=25565, username="N7JumpingBot"):
    """Spawn Mineflayer bot and wait for login event."""
    bot = mineflayer.createBot({
        'host': host,
        'port': port,
        'username': username,
        'hideErrors': False
    })
    once(bot, 'login')
    print(f' Logged in as {username}')
    return bot
```

2.3 Environment definition

Define the environment parameters

[8]:
```
# === Minecraft Jump RL Environment Core (Reward, Done, Teleport/Respawn) ===

# IDs for special blocks from infos.txt
START_BLOCK_ID = 42       # Iron block (start)
PATH_BLOCK_ID = 1         # Stone (path)
CHECKPOINT_BLOCK_ID = 41  # Gold block (checkpoint)
FINISH_BLOCK_ID = 57      # Diamond block (finish)
DEATH_BLOCK_ID = 49       # Obsidian (death)

# Example jump spawn positions (update these for your map as needed)
JUMP_SPAWNS = [
    (8, -55, 12),    # Jump 1
    (-607, 6, -1341)   # Jump 2
]

FINISH_BLOCK_COORDS = [
    (8,-55,-13), # Jump 1
    (8,-55,-13) # Jump 2
]
```

```
Jump1 = {
    "start": (8, -55, 12),
    "finish": (8,-55,-13),
}
Jump2 = {
    "start": (8,-55,-13),
    "finish": (8, -55, 12),
}

JUMPS = [Jump1, Jump2]
```

2.4 Actions definition

[9]:
```
ACTION_NAMES = [
    "forward", "back", "left", "right", "jump", "sprint", "turn_left",␣
 ↪"turn_right"
]
```

2.5 Gym-Style Environment Class

Wrap the Mineflayer bot as a Gym `Env`:
- Describe observation_space (9×6 block matrix)
- Describe action_space (MultiBinary 6 for movement controls)
- List the methods you'll implement: `__init__`, `get_visible_blocks`, `reset`, `step`, `_compute_reward`, `_check_done`, `close`

We create the bot once only.

[10]:
```
import time

def wait_for_bot_connection(host='localhost', port=25565,␣
 ↪username='N7JumpingBot', retry_interval=2):
    """
    Tries to connect the Mineflayer bot repeatedly until success.
    Returns the connected bot object.
    """

    c = 0
    while True:
        try:
            c += 1
            print(c)
            bot = create_bot(host=host, port=port, username=username)
            print("  Bot connected successfully!")
            return bot
        except Exception as e:
            print(f"  Bot connection failed: {e}")
```

```
            print(f"Retrying in {retry_interval} seconds...")
            time.sleep(retry_interval)
            continue

# Usage:
bot = wait_for_bot_connection(host='localhost', port=25565,␣
 ↪username='N7JumpingBot', retry_interval=2)
```

```
1
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
2
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
3
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
4
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
5
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
6
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
7
 Bot connection failed: ('once', 'AggregateError [ECONNREFUSED]: \n    at
internalConnectMultiple (node:net:1139:18)\n    at afterConnectMultiple
(node:net:1712:7)')
Retrying in 2 seconds…
8
 Logged in as N7JumpingBot
 Bot connected successfully!
```

[11]:
```
class MinecraftRL(gym.Env):
    """
    Action space (MultiBinary(8)):
        0: Move forward    --> bot.setControlState('forward', bool(action[0]))
        1: Move back       --> bot.setControlState('back', bool(action[1]))
        2: Strafe left     --> bot.setControlState('left', bool(action[2]))
        3: Strafe right    --> bot.setControlState('right', bool(action[3]))
        4: Jump            --> bot.setControlState('jump', bool(action[4]))
        5: Sprint          --> bot.setControlState('sprint', bool(action[5]))
        6: Turn left       --> bot.entity.yaw -= self.turn_delta if action[6]
        7: Turn right      --> bot.entity.yaw += self.turn_delta if action[7]
    """
    metadata = {'render.modes': ['human']}

    def __init__(self, host='localhost', port=25565, turn_delta=0.15,␣
↪jump_id=0):
        super().__init__()
        self.bot = bot  # Reference the global bot
        self.jump = JUMPS[jump_id]
        self.observation_space = spaces.Box(low=0, high=4096, shape=(9,6),␣
↪dtype=np.int32)
        self.action_space = spaces.MultiBinary(8)
        self.turn_delta = turn_delta
        self.spawn = JUMP_SPAWNS[jump_id]
        self.finish_block_id = FINISH_BLOCK_ID
        self.death_block_id = DEATH_BLOCK_ID
        self.last_distance_to_goal = None

    def teleport_to_start(self):
        x, y, z = self.jump["start"]
        success = False
        attempts = 0
        while not success and attempts < 3:
            try:
                self.bot.chat(f"/tp {int(x)} {int(y)} {int(z)}")
                # You may want to check bot.entity.position after a delay!
                #time.sleep(1)  # Wait for teleport to process
                success = True
            except Exception as e:
                print(f"Teleport attempt {attempts+1} failed: {e}")
                attempts += 1
                time.sleep(0.2)
        if not success:
            print("Teleport to start failed after 3 attempts.")

    def get_visible_blocks(self):
        """
```

```
        Returns (9x6) matrix of block.type in front of the agent.
        If any error occurs, returns zeros.
        """
        try:
            yaw = self.bot.entity.yaw
            px, py, pz = (self.bot.entity.position.x,
                          self.bot.entity.position.y,
                          self.bot.entity.position.z)
            vis = np.zeros((9, 6), dtype=np.int32)
            for d in range(1, 7):
                dx, dz = -math.sin(yaw), math.cos(yaw)
                bx = int(round(px + dx * d))
                bz = int(round(pz + dz * d))
                for i, h in enumerate(range(-4, 5)):
                    by = int(math.floor(py + h))
                    block = self.bot.blockAt(Vec3(bx, by, bz))
                    vis[i, d - 1] = block.type if block else 0
            return vis
        except Exception as e:
            print(f"get_visible_blocks() failed: {e}")
            # Fallback: zeros
            return np.zeros(self.observation_space.shape, dtype=self.
↪observation_space.dtype)


    def agent_on_block(self, block_id):
        # Returns True if agent is standing on block with block_id
        pos = self.bot.entity.position
        block_below = self.bot.blockAt(Vec3(int(pos.x), int(pos.y - 1), int(pos.
↪z)))
        return block_below and block_below.type == block_id

    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        self.teleport_to_start()
        obs = self.get_visible_blocks()
        if obs is None:
            # Fallback: safe value
            obs = np.zeros(self.observation_space.shape, dtype=self.
↪observation_space.dtype)
        self.last_distance_to_goal = self.distance_to_finish()
        return obs, {}


    def distance_to_finish(self):
        pos = self.bot.entity.position
        fx, fy, fz = self.jump["finish"]
```

```
        return ((pos.x - fx)**2 + (pos.y - fy)**2 + (pos.z - fz)**2)**0.5

    def step(self, action):
        """
        Executes the given action using the Mineflayer API.

        action: array-like of shape (8,)
            [forward, back, left, right, jump, sprint, turn_left, turn_right]
        """
        # Movement commands
        self.bot.setControlState('forward', bool(action[0]))
        self.bot.setControlState('back', bool(action[1]))
        self.bot.setControlState('left', bool(action[2]))
        self.bot.setControlState('right', bool(action[3]))
        self.bot.setControlState('jump', bool(action[4]))
        self.bot.setControlState('sprint', bool(action[5]))

        # Turning
        if action[6]:
            self.bot.entity.yaw -= self.turn_delta
        if action[7]:
            self.bot.entity.yaw += self.turn_delta

        time.sleep(0.2)  # Allow action to take effect

        obs = self.get_visible_blocks()
        reward, done = self._compute_reward_done(obs)
        info = {}

        terminated = done
        truncated = False

        return obs, reward, terminated, truncated, info




    def _compute_reward_done(self, obs):
        # Reward and done logic based on agent's position and blocks
        if self.agent_on_block(self.death_block_id):
            # Agent touched a death block (obsidian): episode fails
            reward = -100
            done = True
        elif self.agent_on_block(self.finish_block_id):
            # Agent touched the finish block (diamond): episode succeeds
            reward = +100
            done = True
```

```
        else:
            # Shaping: negative reward for each move, and optionally distance␣
 ↪to goal
            dist = self.distance_to_finish()
            if self.last_distance_to_goal is not None and dist < self.
 ↪last_distance_to_goal:
                reward = +1  # Reward for making progress
            else:
                reward = -1  # Penalty per step if not advancing
            self.last_distance_to_goal = dist
            done = False
        return reward, done

    def close(self):
        self.bot.quit()
```

2.6 Observation Utility Functions

Break out any helper functions used inside your Env, such as:
- Converting world coordinates to agent-relative indices
- Sampling direction vectors from yaw

[12]:
```
from math import sin, cos, floor

def world_to_local(px, py, pz, yaw, depth, height_off):
    """
    Convert a (depth, height) offset in front of the agent into world␣
 ↪coordinates (Vec3).
    - px, py, pz: agent position
    - yaw: agent's rotation in radians
    - depth: how far forward from the agent (in blocks)
    - height_off: vertical offset (in blocks)
    Returns: Vec3 (int x, y, z)
    """
    dx, dz = -sin(yaw), cos(yaw)
    x = int(round(px + dx * depth))
    y = int(floor(py + height_off))
    z = int(round(pz + dz * depth))
    return Vec3(x, y, z)

def get_direction_vector(yaw):
    """
    Returns the unit vector (dx, dz) for the agent's facing direction (yaw in␣
 ↪radians).
    """
    return (-sin(yaw), cos(yaw))
```

2.7 Reward & Done Logic

Outline your reward shaping and terminal conditions:
- Progress reward (e.g. moving closer to goal)
- Penalty for falling or exceeding time
- Done flag when finish line reached or failure

[13]:
```
def compute_progress_reward(prev_vis, curr_vis, goal_block_id=FINISH_BLOCK_ID):
    """
    Reward shaping: +1 for each additional finish block visible compared to␣
 ↪previous observation.
    Use this if you want to encourage the agent to get closer to the goal block.
    """
    return float((curr_vis == goal_block_id).sum() - (prev_vis ==␣
 ↪goal_block_id).sum())

def check_fall_done(bot, death_block_id=DEATH_BLOCK_ID, y_min=-100):
    """
    Checks if the agent has 'fallen':
      - Option 1: if standing on the death block (obsidian)
      - Option 2: y-coordinate below a threshold (e.g., fell into the void)
    Returns: True if episode should be ended due to a fall/death.
    """
    pos = bot.entity.position
    block_below = bot.blockAt(Vec3(int(pos.x), int(pos.y - 1), int(pos.z)))
    # Use block_below.type == death_block_id for map-specific death logic
    on_death_block = block_below and block_below.type == death_block_id
    below_min_y = pos.y < y_min
    return on_death_block or below_min_y
```

2.8 Visualization Helpers

Provide functions to help you debug and visualize:
- Plotting the block-ID matrix as a heatmap
- Rendering a simple textual or graphical view of the agent's surroundings

[14]:
```
import matplotlib.pyplot as plt

def plot_block_matrix(vis):
    """Heatmap of block IDs (default colormap)."""
    plt.figure()
    plt.imshow(vis, origin='lower', aspect='auto')
    plt.colorbar(label='Block ID')
    plt.xlabel('Depth (blocks ahead)')
    plt.ylabel('Height offset (-4..+4)')
    plt.title('Visible Blocks')
    plt.show()
```

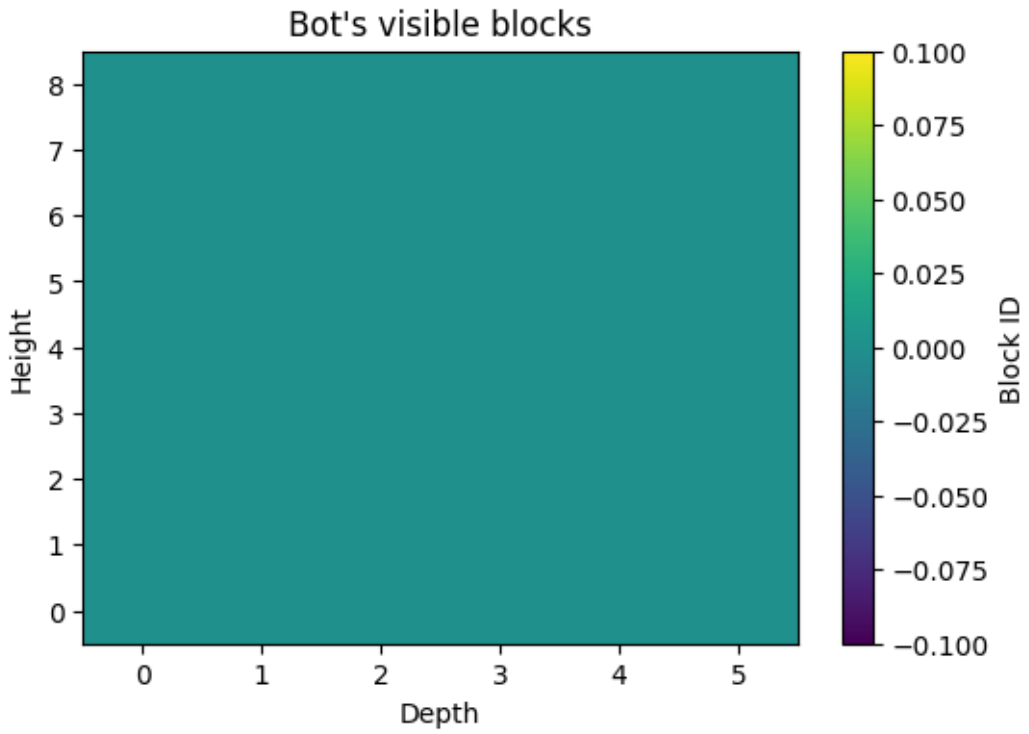
```
def render_agent_view(env):
    """Quick render of the agent's current visible blocks."""
    vis = env.get_visible_blocks()
    plot_block_matrix(vis)
```

3 Test code to check if functions are working correctly

[15]:
```
# 1. Instantiate your environment (do this after the bot is created and␣
 ↪connected)
env = MinecraftRL(jump_id=0)  # or jump_id=1 for Jump2

# 2. Teleport to start and print position
print("Teleporting to start position...")
env.teleport_to_start()
#time.sleep(2)  # Wait for teleport to process

pos = env.bot.entity.position
print(f"Bot position: ({pos.x:.2f}, {pos.y:.2f}, {pos.z:.2f})")

# 3. Test observations: get and display the block matrix
obs = env.get_visible_blocks()
print("Observation matrix shape:", obs.shape)
print("Block matrix (IDs):")
print(obs)

# Optional: Visualize
import matplotlib.pyplot as plt
plt.figure(figsize=(6,4))
plt.imshow(obs, origin='lower', aspect='auto')
plt.colorbar(label='Block ID')
plt.title("Bot's visible blocks")
plt.xlabel('Depth')
plt.ylabel('Height')
plt.show()

# 4. Test action execution: try moving forward & jumping
print("Testing forward movement and jump...")

# Format: [fwd, back, left, right, jump, sprint, turnL, turnR]
move_forward = [1,0,0,0,0,0,0,0]
jump_only    = [0,0,0,0,1,0,0,0]
jump_forward = [1,0,0,0,1,0,0,0]

# Do one action at a time
for i, action in enumerate([move_forward, jump_only, jump_forward]):
    print(f"Applying action {i}: {action}")
```

```
    obs2, reward, terminated, truncated, info = env.step(action)
    print("Bot pos:", env.bot.entity.position)
    print("Reward:", reward, "Terminated:", terminated, "Truncated:", truncated)
    print("Block matrix (IDs):")
    print(obs2)
    time.sleep(2)  # Wait so you can observe in Minecraft if you want

print("All tests completed.")
```

```
Teleporting to start position…
Bot position: (8.50, -56.00, 13.70)
Observation matrix shape: (9, 6)
Block matrix (IDs):
[[0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]
 [0 0 0 0 0 0]]
```




```
Testing forward movement and jump…
```

```
Applying action 0: [1, 0, 0, 0, 0, 0, 0, 0]
Bot pos: Vec3 { x: 8.5, y: -56,
z: 13.7 }
Reward: -1 Terminated: False Truncated: False
Block matrix (IDs):
[[  0 163 195   0   0   0]
 [  0 163 195   0   0   0]
 [  0 163 195   0   0   0]
 [163 163 195   0   1   1]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]]
Applying action 1: [0, 0, 0, 0, 1, 0, 0, 0]
Bot pos: Vec3 { x: 8.5, y:
-54.975575911786315, z: 13.7 }
Reward: -1 Terminated: False Truncated: False
Block matrix (IDs):
[[  0   0 163 195   0   0]
 [  0   0 163 195   0   0]
 [163 163 163 195   0   1]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]]
Applying action 2: [1, 0, 0, 0, 1, 0, 0, 0]
Bot pos: Vec3 { x: 8.5, y:
-55.2032643993313, z: 13.7 }
Reward: -1 Terminated: False Truncated: False
Block matrix (IDs):
[[  0   0 163 195   0   0]
 [  0   0 163 195   0   0]
 [163 163 163 195   0   1]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]
 [  0   0   0   0   0   0]]
All tests completed.
```

[16]:
```python
import time
print("Moving forward for 2 seconds...")
bot.setControlState('forward', True)
```

```
time.sleep(2)
bot.setControlState('forward', False)
print("Did the bot move?")
```

```
Moving forward for 2 seconds…
Did the bot move?
```

3.1 Wrapping with Stable-Baselines3

Explain how to instantiate an RL agent:
- Choose PPO (or DQN)
- Pass your `MinecraftRL` env into the model

[17]:
```
from stable_baselines3 import PPO
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.env_util import make_vec_env

log_dir = "./logs/"
os.makedirs(log_dir, exist_ok=True)
# Make a non-vectorized env (or set n_envs=1)
monitored_env = Monitor(MinecraftRL(), log_dir)

model = PPO("MlpPolicy", monitored_env, verbose=1, n_steps=2048)
```

```
Using cpu device
Wrapping the env in a DummyVecEnv.
```

3.2 Training Loop

Describe running `model.learn`:
- Total timesteps or episodes
- Where you log rewards/metrics

[18]:
```
from stable_baselines3.common.callbacks import CheckpointCallback

# save every 10k steps
checkpoint_cb = CheckpointCallback(save_freq=10_000, save_path=log_dir,␣
 ↪name_prefix="ppo_mc")
model.learn(total_timesteps=200_000, callback=checkpoint_cb)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[18], line 5
      3 # save every 10k steps
      4 checkpoint_cb = CheckpointCallback(save_freq=10_000, save_path=log_dir,
 ↪name_prefix="ppo_mc")
----> 5 model.learn(total_timesteps=200_000, callback=checkpoint_cb)
```

```
File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\ppo\ppo.
 ↪py:311, in PPO.learn(self, total_timesteps, callback, log_interval,␣
 ↪tb_log_name, reset_num_timesteps, progress_bar)
    302 def learn(
    303     self: SelfPPO,
    304     total_timesteps: int,
   (…)    309     progress_bar: bool = False,
    310 ) -> SelfPPO:
--> 311     return super().learn(
    312         total_timesteps=total_timesteps,
    313         callback=callback,
    314         log_interval=log_interval,
    315         tb_log_name=tb_log_name,
    316         reset_num_timesteps=reset_num_timesteps,
    317         progress_bar=progress_bar,
    318     )

File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\on_pol
 ↪py:324, in OnPolicyAlgorithm.learn(self, total_timesteps, callback,␣
 ↪log_interval, tb_log_name, reset_num_timesteps, progress_bar)
    321 assert self.env is not None
    323 while self.num_timesteps < total_timesteps:
--> 324     continue_training =␣
 ↪self.collect_rollouts(self.env, callback, self.rollout_buffer, n_rollout_steps=self.n_steps
    326     if not continue_training:
    327         break

File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\on_pol
 ↪py:218, in OnPolicyAlgorithm.collect_rollouts(self, env, callback,␣
 ↪rollout_buffer, n_rollout_steps)
    213     else:
    214         # Otherwise, clip the actions to avoid out of bound error
    215         # as we are sampling from an unbounded Gaussian distribution
    216         clipped_actions = np.clip(actions, self.action_space.low, self.
 ↪action_space.high)
--> 218 new_obs, rewards, dones, infos = env.step(clipped_actions)
    220 self.num_timesteps += env.num_envs
    222 # Give access to local variables

File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\vec_en
 ↪py:222, in VecEnv.step(self, actions)
    215 """
    216 Step the environments with the given action
    217
    218 :param actions: the action
```

```
    219 :return: observation, reward, done, information
    220 """
    221 self.step_async(actions)
--> 222 return self.step_wait()

File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\vec_en
 ↪py:59, in DummyVecEnv.step_wait(self)
     56 def step_wait(self) -> VecEnvStepReturn:
     57     # Avoid circular imports
     58     for env_idx in range(self.num_envs):
---> 59         obs, self.buf_rews[env_idx], terminated, truncated, self.
 ↪buf_infos[env_idx] = self.envs[env_idx].step(  # type: ignore[assignment]
     60             self.actions[env_idx]
     61         )
     62         # convert to SB3 VecEnv api
     63         self.buf_dones[env_idx] = terminated or truncated

File␣
 ↪~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\monito
 ↪py:94, in Monitor.step(self, action)
     92 if self.needs_reset:
     93     raise RuntimeError("Tried to step environment that needs reset")
---> 94 observation, reward, terminated, truncated, info = self.env.step(action
     95 self.rewards.append(float(reward))
     96 if terminated or truncated:

Cell In[11], line 113, in MinecraftRL.step(self, action)
    110 if action[7]:
    111     self.bot.entity.yaw += self.turn_delta
--> 113 time.sleep(0.2)  # Allow action to take effect
    115 obs = self.get_visible_blocks()
    116 reward, done = self._compute_reward_done(obs)

KeyboardInterrupt:
```

3.3 Training Logs & Plots

Show how to load or collect your training logs and plot the learning curve:
- Use Matplotlib to plot episode returns over time

[ ]:
```
import pandas as pd

df = pd.read_csv(os.path.join(log_dir, "monitor.csv"), skiprows=1)
# rolling mean of reward
df['r_mean'] = df['r'].rolling(10).mean()
```

```
plt.figure()
plt.plot(df['r_mean'])
plt.xlabel("Episode")
plt.ylabel("Rolling Mean Reward")
plt.show()
```

3.4 Evaluation & Demo Episodes

Roll out a few episodes under the trained policy:
- Reset env, step with `model.predict`
- Use your visualization helpers to display agent behavior

[ ]:
```
from stable_baselines3.common.evaluation import evaluate_policy

# Automated eval
mean_reward, std_reward = evaluate_policy(model, monitored_env,␣
 ↪n_eval_episodes=5)
print(f"Mean reward: {mean_reward:.2f} +/- {std_reward:.2f}")

# Manual demo
for ep in range(3):
    obs = monitored_env.reset()
    done = False
    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, done, _ = monitored_env.step(action)
        render_agent_view(monitored_env)
```

3.5 Hyperparameter Tuning Tips

List important hyperparameters to experiment with:
- Learning rate, gamma, clip range, batch size, etc.
- Suggest grid-search or Bayesian tuning approaches

[ ]:
```
param_grid = {
    'learning_rate': [1e-4, 3e-4, 1e-3],
    'gamma': [0.95, 0.99, 0.999],
    'n_steps': [128, 256, 512],
}
print("Example grid:", param_grid)
```

3.6 Conclusion & Next Steps

Summarize results and propose extensions:
- Continuous yaw control
- Curriculum learning on progressively harder course
- Integration with more complex objectives like doing courses using different mechanics like using a ladder on the side of a block, having an obstacle moving on a jump trajectory and more complex

blocks arrangments needing to guess the good way when there are many possible next jump choice possible.

[ ]:
```
bot.quit()
```

[ ]:
```

```